HTTP error handling with JavaScript

Status codes, Fetch API, Axios, and resilient client design

Where HTTP errors happen

1

Request

URL, method, headers, body

→

2

Transport

DNS, TLS, connection, timeout

→

3

HTTP response

Status, headers, response body

→

4

Application

Interpret result and update UI

HTTP error: a response arrived with a status such as 404 or 500.

Network error: no usable HTTP response was received.

The distinction determines which code path should run.

HTTP status code families

1xx	**Informational**	Interim protocol information
2xx	**Success**	The request was successfully received / understood
3xx	**Redirection**	Further action may be needed
4xx	**Client error**	The request cannot be fulfilled as sent
5xx	**Server error**	The server failed to fulfill a valid request

Clients should usually branch on status - not brittle message text.

Common statuses in client applications

200 OK	Request succeeded; response may contain data
201 Created	A resource was created
204 No Content	Success with no response body
400 Bad Request	Malformed or invalid request
401 Unauthorized	Authentication is missing or invalid
403 Forbidden	Authenticated or not, access is not permitted
404 Not Found	Resource or route was not found
409 Conflict	Request conflicts with current resource state
422 Unprocessable Content	Validation or semantic errors (API-dependent)
429 Too Many Requests	Rate limit; inspect Retry-After if supplied
500 / 503	Server failure / temporarily unavailable

Exact status usage varies by API; follow the server's documented contract.

Fetch: check response.ok explicitly

```
async function getOrder(id) {
  const response = await fetch(`/api/orders/${id}`);
  if (!response.ok) {
    throw new Error(
      `HTTP ${response.status}`
    );
  }
  if (response.status === 204) return null;
  return response.json();
}
try {
  const order = await getOrder(42);
  renderOrder(order);
} catch (error) {
  showLoadFailure();
  console.error(error);
}
```

Why the check matters

fetch() normally resolves for HTTP 404 and 500 responses.

response.ok is true for statuses 200–299.

Throwing here converts an unacceptable status into a rejected async flow that catch can handle.

204 responses have no JSON body to parse.

Axios: non-2xx rejects by default

```
import axios from "axios";

async function getOrder(id) {
  try {
    const { data } = await axios.get(
      `/api/orders/${id}`,
      { timeout: 5000 }
    );
    return data;
  } catch (error) {
    if (axios.isAxiosError(error)) {
      console.log(error.response?.status);
      console.log(error.response?.data);
    }
    throw error;
  }
}
```

Axios error anatomy

error.response: server replied with a status rejected by validateStatus.

error.request: request was sent but no response arrived.

Neither: request setup failed.

error.response.data often contains the API's error payload.

Do not assume the server always returns JSON.

Fetch vs Axios: HTTP status behavior

CASE	FETCH	AXIOS DEFAULT
200 OK	Promise resolves	Promise resolves
404 Not Found	Promise resolves; inspect ok	Promise rejects; catch runs
500 Server Error	Promise resolves; inspect ok	Promise rejects; catch runs
Offline / DNS failure	Promise rejects	Promise rejects
Parse / transform error	response.json() can reject	Transforming data may throw / reject

Axios behavior can be customized with validateStatus.

Read structured error responses

```
try {
  const { data } = await axios.post(
    "/api/users",
    { email: "not-an-email" }
  );
  return data;
} catch (error) {
  if (axios.isAxiosError(error)) {
    const status = error.response?.status;
    const details = error.response?.data;
   if (status === 422) {
      showValidationErrors(details);
      return;
    }
  }
  showGenericFailure();
}
```

API contract

Agree with the backend on a predictable error shape, for example:

{ code, message, details }

Use status for broad control flow and structured fields for specific validation messages.

Validate data on the server even if the client validates too.

Centralize status handling

```
async function requestJson(url) {
  const response = await fetch(url, {
    headers: { Accept: "application/json" }
  });

  const body = response.status === 204
    ? null
    : await response.json();

  if (!response.ok) {
    throw new HttpError(
      response.status,
      body
    );
  }
  return body;
}
```

Why centralize?

- Avoid repeating status checks.
- Normalize error formats across endpoints.
- Keep UI components focused on user experience.
- Add request IDs or diagnostic context in one place.

Define HttpError as a custom Error subclass in production code.

Retries: when are they appropriate?

Potentially retryable

- Temporary network failure
- 503 Service Unavailable
- 429 Too Many Requests, respecting Retry-After
- Use bounded attempts and backoff

Do not blindly retry

- 400 validation errors
- 401 / 403 without changing credentials or permissions
- Non-idempotent POST that may have succeeded server-side
- Every error forever

Use idempotency keys (UUID) where supported; a timeout does not prove the server did nothing.

Map statuses to user experience

400 / 422	Show actionable field-level validation feedback
401	Sign in again or refresh credentials where appropriate
403	Explain that the action is not permitted
404	Show not-found state or navigate to a safe fallback
429	Explain rate limiting; wait according to API guidance
5xx / network	Offer retry where sensible and preserve user input

Never display raw stack traces, credentials, or untrusted server internals to end users.

Common HTTP handling mistakes

- ✗ Assuming fetch() throws for 404 or 500.
- ✗ Treating all errors as server errors; network and setup failures differ.
- ✗ Parsing JSON unconditionally when a 204 response has no body.
- ✗ Depending on a human-readable error message instead of status / error code.
- ✗ Retrying writes without considering duplicate side effects.
- ✗ Logging tokens, personal data, or raw sensitive server responses.

Practical excercise: build a status-aware client

```
async function createBooking(payload) {
  const response = await fetch('/api/bookings', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });

  // TODO: handle 201, 400/422, 401, 429, 500
  // TODO: don't parse JSON for 204
  return response.json();
}
```

Pair discussion

1. Which statuses are expected outcomes?

2. Which should reject a shared helper?

3. What payload should the UI show for 422?

4. Is retry safe after a timeout on POST?

5. Rewrite with Axios.

Takeaways & references

1. HTTP status and network failure are different categories.
2. Fetch requires explicit response.ok / status handling.
3. Axios rejects non-2xx responses by default; inspect error.response.
4. Use status codes for broad decisions and structured payloads for details.
5. **Centralize policies; retry carefully; present safe, useful feedback.**

References: MDN ->HTTP response status codes and Fetch API • Axios documentation -> Handling Errors